

HW #3

a)

$$1. 0x14F = 335$$

$$t = (335 + 1) \left(\frac{1}{52768} \right) = 0.0102539$$

$$b) 0.005 \geq |x_{int} - 0.01 - x_{int} - 0.0102539|$$

$$0.005 \geq |x_{int}(0.01 - 0.0102539)|$$

$$x_{int} = \frac{0.005}{0.0002539} = 19.69 \approx 20 \text{ interrupts}$$

& we rounded up for interrupts

because the actual time is greater than 0.01, the system is fast

$$c) \text{int_leapCount} = 0; \text{int_interruptCount} = 0;$$

```
#pragma vector=TIMER2_A0_VECTOR
```

```
__interrupt void TIMER_A2_ISR(void) {
```

```
    if (leapCount < 20) {
```

```
        interruptCount++;
```

```
        leapCount;
```

```
    } else {
```

```
        interruptCount += 2;
```

```
        leapCount = 0;
```

```
    }
```

```
}
```

```
-BIS_SR(GIE)
```

$$d) 1.048576 \text{ MHz} = 0.9536 \mu\text{s}$$

$$\frac{0.01}{0.9536 \mu\text{s}} = 10486 \text{ cycles}$$

it is not a good idea to set the timer at that interval as it'll consume a lot processing power

2. a) $\frac{5,508,239,000 \text{ s}}{60 \text{ s}} \bigg| \frac{1 \text{ min}}{60 \text{ min}} \bigg| \frac{1 \text{ hr}}{24 \text{ hr}} \bigg| \frac{1 \text{ day}}{7 \text{ days}} = 8776 \text{ weeks}$
 $R = 514200 \text{ s}$
 $\frac{8776 \text{ weeks}}{52 \text{ weeks}} \bigg| \frac{1 \text{ yr}}{1 \text{ yr}} = 168 \text{ yrs}$ $R 40 \text{ weeks} = 10 \text{ weeks}$
 $\frac{514200 \text{ s}}{60 \text{ s}} \bigg| \frac{1 \text{ min}}{60 \text{ min}} \bigg| \frac{1 \text{ hr}}{24 \text{ hr}} = 6 \text{ days}$
 $R = 1 \text{ hr}$ $R = 30 \text{ mins}$ $R = 40 \text{ s}$

01:36:40 on 6 October 2068

b) $\frac{5,496,386 \text{ s}}{60 \text{ s}} \bigg| \frac{1 \text{ min}}{60 \text{ min}} \bigg| \frac{1 \text{ hr}}{24 \text{ hr}} \bigg| \frac{1 \text{ day}}{7 \text{ days}} = 9 \text{ weeks} = 2 \text{ months} \&$
 $R = 13156 \text{ s}$ 7 days
 $\frac{13156 \text{ s}}{60 \text{ s}} \bigg| \frac{1 \text{ min}}{60 \text{ min}} \bigg| \frac{1 \text{ hr}}{1 \text{ hr}} = 3 \text{ hrs}$
 $R = 39 \text{ min}$ $R = 16 \text{ s}$

February 7, 03:59:16

2262-04-12

c) the largest number in a 64-bit int is 9,223,372,036,854,775,807, and the number of nanoseconds in a year is 31,557,000,000,000,000, meaning that it can be stored in a 64-bit unsigned int

d) `uint32_t utc = 0;`

```
void runTimerA2(void) {
```

```
    TACTL = TASEL_1 + IB_3 + MC_1; // use MCLK
```

```
    TA2CCR0 = 1048575 // (max_count + 1) *  $\frac{1}{1.04 \text{ MHz}}$  = 1
```

```
    TA2CTL = CCIE;
```

```
}
```

```
void stopTimerA2(void) {
```

```
    TACTL = MC_0;
```

```
    TA2CTL &= ~CCIE;
```

```
    utc = 0;
```

```
}
```

```
#pragma vector = TIMER2_A0_VECTOR
```

```
__interrupt void TimerA2_ISR(void) {
```

```
    utc++; // no need for leap count because it's exact already
```

```
}
```

3. a) void configureMCS() {

 PSEL = 00001100;

 MCSCTL6 &= ~XT2OFF;

 MCSCTL3 |= SELREF_2;

 MCSCTL4 |= SELA_2;

 REF0-bis_SR_register(SCG0);

 MCSCTL0 = 00000000;

 MCSCTL1 = DCORSEL_7;

 MCSCTL2 = FLLB_1 + 762;

 __bic_SR_register(SCG0);

 settle_delay_cycles(250000);

 while (SFRIFG1 & OFIFG) {

 MCSCTL7 &= ~(XT2OFFG + XT1LFOFFG + XT1HFOFFG + DCOFFG);

 SFRIFG1 &= ~OFIFG;

 }

 MCSCTL4 |= SELS_5 + SELM_5;

}

b) void configureTimerA2() {

 TA2CTL = TASSEL_2 + ID_1 + MC_1;

 TA2CCTL0 = CCIE;

 TA2CCR0 = 8000; // no need for leap count b/c exact

}

#pragma vector=TIMER_A0_VECTOR

__interrupt void Timer_A2_ISR(void) { // do something }

c) there's no need for leap count as it is within range of where the timer is accurate

d) 2

dc[-1]

2